Java基础篇

① Java特点

- 平台无关性
  - Java编译器将源代码编译成字节码，字节码可以在任何安装了JVM的系统上运行。
- 面向对象
  - Java是一门严格的面向对象编程语言，几乎一切都是对象。
  - 类（class）、对象（object）、继承（inheritance）、多态（polymorphism）、抽象（abstraction）和封装（encapsulation）
  - 三大特性
    - 封装
      - 将对象的属性（数据）和行为（方法）结合在一起，对外隐藏对象的内部细节，仅通过对象提供的接口与外界交互
      - 增强安全性和简化编程，使得对象更加独立。
    - 继承
      - 可以使得子类自动共享父类数据结构和方法的机制
      - 代码复用的重要手段
    - 多态
      - 同一操作作用于不同对象时，可以有不同的解释和行为
        - 方法重载
          - 同一类中可以有多个同名方法，它们具有不同的参数列表（参数类型、数量或顺序不同）
          - 虽然方法名相同，但根据传入的参数不同，编译器会在编译时确定调用哪个方法。
        - 方法重写
          - 子类能够提供对父类中同名方法的具体实现
          - 在运行时，JVM会根据对象的实际类型确定调用哪个版本的方法
        - 接口与实现
          - 多个类可以实现同一个接口，并且用接口类型的引用来调用这些类的方法。
        - 向上转型和向下转型
          - 向上转型
            - 使用父类类型的引用指向子类对象
          - 向下转型
            - 将父类引用转回其子类类型
      - 子类可以替换父类，在实际的代码运行过程中，调用子类的方法实现。
        - 提高代码的扩展性和复用性，是很多设计模式、设计原则、编程技巧的代码实现基础。
  - 设计原则
    - 单一职责原则
      - 一个类应该只负责一项职责
    - 开放封闭原则
      - 软件实体应该对扩展开放，对修改封闭
    - 里氏替换原则
      - 子类对象应该能够替换掉所有父类对象
    - 接口隔离原则
      - 客户端不应该依赖那些它不需要的接口
    - 依赖倒置原则
      - 高层模块不应该依赖低层模块，二者都应该依赖于抽象；抽象不应该依赖于细节，细节应该依赖于抽象。
    - 最少知识原则
      - 一个对象应当对其他对象有最少的了解，只与其直接的朋友交互
- 内存管理
  - Java有自己的垃圾回收机制，自动管理内存和回收不再使用的对象

① 泛型

- 允许类、接口和方法在定义时使用一个或多个类型参数，这些类型参数在使用时可以被指定为具体的类型。
  - 主要目的是在编译时提供更强的类型检查，并且在编译后能够保留类型信息，避免了在运行时出现类型转换异常。
- 为什么需要
  - 适用于多种数据类型执行相同的代码
  - 泛型中的类型在使用时指定，不需要强制类型转换（类型安全，编译器会检查类型）

① 反射

- 特性
  - 在运行状态中，对于任意一个类，都能够知道这个类中的所有属性和方法，对于任意一个对象，都能够调用它的任意一个方法和属性；这种动态获取的信息以及动态调用对象的方法的功能称为 Java 语言的反射机制。
  - 运行时类信息访问
    - 允许程序在运行时获取类的完整结构信息，包括类名、包名、父类、实现的接口、构造函数、方法和字段等。
  - 动态对象创建
    - 使用反射API动态地创建对象实例，即使在编译时不知道具体的类名。
  - 动态方法调用
    - 可以在运行时动态地调用对象的方法，包括私有方法。这通过Method类的invoke()方法实现，允许你传入对象实例和参数值来执行方法。
  - 访问和修改字段值
    - 反射还允许程序在运行时访问和修改对象的字段值，即使是私有的。这是通过Field类的get()和set()方法完成的。
- 应用场景
  - 加载数据库驱动
    - 我们在使用 JDBC 连接数据库时使用 Class.forName()通过反射加载数据库的驱动程序，如果是mysql则传入mysql的驱动类，而如果是oracle则传入的参数就变成另一个了。
  - 配置文件加载
    - Spring 框架的 IOC（动态加载管理 Bean），Spring通过配置文件配置各种各样的bean，你需要用到哪些bean就配哪些，spring容器就会根据你的需求去动态加载
      - Spring通过XML配置模式装载Bean的过程
        - 将程序中所有XML或properties配置文件加载入内存
        - Java类里面解析xml或者properties里面的内容，得到对应实体类的字节码字符串以及相关的属性信息
        - 使用反射机制，根据这个字符串获得某个类的Class实例
        - 动态配置实例的属性

① 数据类型

- 基本数据类型
  - 数值型
    - 整数类型
      - byte、short、int、long
    - 浮点类型
      - float、double
        - Double和BigDecimal
          - Double
            - 二进制浮点运算
              - 二进制有些情况下不能准确的表示一个小数
          - BigDecimal
            - 十进制浮点运算
              - 使用字符串作为参数，以避免浮点数精度丢失。
  - 字符型
  - 布尔型
- 引用数据类型
  - 类
  - 接口
  - 数组
- 类型转换
  - 方式
    - 自动类型转换（隐式转换）
      - 当目标类型的范围大于源类型时，Java会自动将源类型转换为目标类型，不需要显式的类型转换。
      - int转换为long
    - 强制类型转换（显式转换）
      - 当目标类型的范围小于源类型时，需要使用强制类型转换将源类型转换为目标类型。这可能导致数据丢失或溢出。
      - long转换为int
    - 字符串转换
      - Java提供了将字符串表示的数据转换为其他类型数据的方法。
      - Integer.parseInt()
    - 数值之间的转换
      - Java提供了一些数值类型之间的转换方法，如将整型转换为字符型、将字符型转换为整型等。
  - 问题
    - 数据丢失
      - 将一个范围较大的数据类型转换为一个范围较小的数据类型时，可能会发生数据丢失。
      - 转换结果将是截断后的低位部分，高位部分的数据将丢失。
    - 精度丢失
      - 浮点数转换为整数时，小数部分会被截断，导致精度丢失。
    - 类型不兼容
      - 尝试将不兼容的类型进行转换时，可能会导致编译错误或运行时错误。
- 包装类
  - 为什么需要
    - 泛型中的应用
      - 泛型只能使用引用类型，而不能使用基本类型
      - 例如集合只能存储包装类
    - 转换中的应用
      - 基本类型和引用类型不能直接进行转换，必须使用包装类来实现
      - String str = integer.toString();
  - 区别
    - 基本数据类型
      - 变量对应的内存块直接存储数据本身
      - 不需要实例化就可以使用
      - int变量可以直接赋值为0
      - 基本数据类型用于高性能、低内存占用的场景
        - 性能
          - 基本数据类型的操作直接在栈上进行，效率非常高
        - 内存占用
          - 基本数据类型占用的内存非常小（例如 int 占 4 字节，double 占 8 字节），而包装类是对象，需要额外的内存开销（例如对象头、引用等）
    - 包装类
      - 对象的引用和对象本身是分开存储的
      - 通过实例化对象来使用
      - Integer变量必须通过实例化对象来赋值
        - 如果对一个未经初始化的Integer变量进行操作，就会出现空指针异常。
      - Integer的缓存
        - 内部实现了一个静态缓存池，用于存储特定范围内的整数值对应的Integer对象。
        - 默认情况下，这个范围是-128至127
        - 通过Integer.valueOf(int)方法创建一个在这个范围内的整数对象时，并不会每次都生成新的对象实例，而是复用缓存中的现有对象，会直接从内存中取出，不需要新建一个对象。
      - 包装类用于需要对象化操作的场景
  - 装箱和拆箱
    - 装箱（Boxing）和拆箱（Unboxing）是将基本数据类型和对应的包装类之间进行转换的过程。
    - 自动装箱
      - 发生情况
        - 赋值时
          - Integer iObject = 3; //autobxing - primitive to wrapper conversion
        - 方法调用时
      - 弊端
        - 在一个循环中进行自动装箱操作的情况，会创建多余的对象

① 普通类和抽象类和接口

- 普通类
  - 可以直接实例化对象
  - 方法可以有具体的实现
- 抽象类
  - 用于描述类的共同特性和行为，可以有成员变量、构造方法和具体方法。适用于有明显继承关系的场景。
    - 不能被实例化，只能被继承
    - 方法可以有实现也可以没有实现
    - 只能继承一个抽象类
    - 关键字为extends
    - 可以包含实例变量和静态变量
    - 可以有构造器，这些构造器在子类实例化时会被调用，以便进行必要的初始化工作
  - 抽象类和final修饰符是互斥的
    - final修饰符用于禁止类被继承或方法被重写
- 接口
  - 用于定义行为规范，可以多实现，只能有常量和抽象方法（Java 8以后可以有默认方法和静态方法）。适用于定义类的能力或功能。
    - 不能被实例化，只能被实现
    - 只有定义，不能有方法的实现，java 1.8中可以定义default方法体
      - 抽象方法
        - 默认是 public 和 abstract，这些修饰符可以省略
        - void makeSound();
      - 默认方法
        - 需要实现类的对象才能调用
        - default void sleep() {System.out.println("Sleeping...");}
      - 静态方法
        - Java 8 中引入，属于接口本身，只能通过接口名直接调用，不需要实现类的对象。
      - 私有方法
        - Java 9 中引入，用于在接口中为默认方法或其他私有方法提供辅助功能
    - 关键字为implements
    - 可以实现多个接口
    - 只能包含常量（即静态常量）
    - 不可以有构造方法
      - 接口不会有自己的实例的，所以不需要有构造函数

① 深拷贝和浅拷贝

- 浅拷贝
  - 只复制对象本身和其内部的值类型字段，但不会复制对象内部的引用类型字段。
  - 如果原对象内部有引用类型的字段，只是将引用复制到新对象中，两个对象指向的是同一个引用对象。
- 深拷贝
  - 在复制对象的同时，将对象内部的所有引用类型字段的内容也复制一份，而不是共享引用。
  - 会递归复制对象内部所有引用类型的字段，生成一个全新的对象以及其内部的所有对象。
  - 实现方法
    - 实现 Cloneable 接口并重写 clone() 方法
      - 这种方法要求对象及其所有引用类型字段都实现 Cloneable 接口，并且重写 clone() 方法。在 clone() 方法中，通过递归克隆引用类型字段来实现深拷贝。
    - 使用序列化和反序列化
      - 通过将对象序列化为字节流，再从字节流反序列化为对象来实现深拷贝。要求对象及其所有引用类型字段都实现 Serializable 接口。
    - 手动递归复制
      - 针对特定对象结构，手动递归复制对象及其引用类型字段。适用于对象结构复杂度不高的情况。

① 对象

- 创建方式
  - new关键字
    - 通过new关键字直接调用类的构造方法来创建对象。
    - MyClass obj = new MyClass();
  - 通过反射创建对象
    - Class类的newInstance()方法
      - 通过反射机制，可以使用Class类的newInstance()方法创建对象。
    - Constructor类的newInstance()方法
      - 通过反射机制，可以使用Constructor类的newInstance()方法创建对象。
  - clone()方法
    - 通过将对象序列化（保存到文件或网络传输）然后再反序列化（从文件或网络传输中读取对象）的方式来创建对象
  - 反序列化
    - 通过将对象序列化（保存到文件或网络传输）然后再反序列化（从文件或网络传输中读取对象）的方式来创建对象
- New出的对象什么时候回收？
  - 引用计数法
    - 某个对象的引用计数为0时，表示该对象不再被引用，可以被回收。
  - 可达性分析算法
    - 从根对象（如方法区中的类静态属性、方法中的局部变量等）出发，通过对象之间的引用链进行遍历，如果存在一条引用链到达某个对象，则说明该对象是可达的，反之不可达，不可达的对象将被回收。
  - 终结器（Finalizer）
    - 如果对象重写了finalize()方法，垃圾回收器会在回收该对象之前调用finalize()方法，对象可以在finalize()方法中进行一些清理操作。
- == 与 equals
  - ==
    - 比较的是两个字符串内存地址（堆内存）的数值是否相等，属于数值比较
  - equals
    - 比较的是两个字符串的内容，属于内容比较
    - 对于非字符串变量来说，如果没有对equals()进行重写的话，"==" 和 "equals"方法的作用是相同的
- StringBuilder和StringBuffer
  - String
    - 是 Java 中基础且重要的类，被声明为 final class，是不可变字符串
      - 因为它的不可变性，所以拼接字符串时候会产生很多无用的中间对象，如果频繁的进行这样的操作对性能有所影响。
    - 为什么设计为不可变？
      - 安全性
        - 不可变性确保了 String 对象的内容不能被意外或恶意修改
      - 线程安全
        - 不可变对象天生是线程安全的，因为它们的状态无法被修改。多个线程可以共享同一个 String 对象，而不需要额外的同步机制。
      - 性能优化
        - 不可变性使得 String 可以进行一些性能优化，例如字符串池（String Pool）和缓存哈希值。
  - StringBuilder
    - 和 StringBuffer 本质上没什么区别，就是去掉了保证线程安全的那部分，减少了开销。
    - 线程不安全
  - StringBuffer
    - 为了解决大量拼接字符串时产生很多中间对象问题而提供的一个类。
    - 本质是一个线程安全的可修改的字符序列
  - 速度从快到慢为 StringBuilder > StringBuffer > String

② 静态变量和静态方法

- 静态变量
  - 使用static关键字声明的变量属于类而不是任何具体的对象
    - 共享性
      - 所有该类的实例共享同一个静态变量
    - 初始化
      - 静态变量在类被加载时初始化，只会对其进行一次分配内存
    - 访问方式
      - 静态变量可以直接通过类名访问，也可以通过实例访问，但推荐使用类名
  - 使用场景
    - 常用于需要在所有对象间共享的数据，如计数器、常量等。
- 静态方法
  - 使用static关键字声明的方法静态方法也属于类，而不是任何具体的对象
    - 无实例依赖
      - 可以在没有创建类实例的情况下调用。
    - 访问静态成员
      - 可以直接调用其他静态变量和静态方法，但不能直接访问非静态成员。
    - 多态性
      - 不支持重写（Override），但可以被隐藏（Hide）
  - 使用场景
    - 常用于助手方法（utility methods）、获取类级别的信息或者是没有依赖于实例的数据处理。
- 非静态内部类和静态内部类
  - 非静态内部类
    - 依赖于外部类的实例
      - 可以访问外部类的实例变量和方法
        - 编译器在生成字节码时会为非静态内部类维护一个指向外部类实例的引用。
        - 编译器会在生成非静态内部类的构造方法时，将外部类实例作为参数传入，并在内部类的实例化过程中建立外部类实例与内部类实例之间的联系，从而实现直接访问外部方法的功能。
    - 不能定义静态成员
    - 在外部类实例化后才能实例化
    - 可以访问外部类的私有成员
  - 静态内部类
    - 不依赖于外部类的实例
    - 只能访问外部类的静态成员
    - 可以定义静态成员
    - 可以独立实例化
    - 不能直接访问外部类的私有成员，需要通过实例化外部类来访问。

② 序列化

- 怎么把一个对象从一个jvm转移到另一个jvm？
  - 使用序列化和反序列化
    - 将对象序列化为字节流，并将其发送到另一个 JVM，然后在另一个 JVM 中反序列化字节流恢复对象。
  - 使用消息传递机制
    - 利用消息传递机制，比如使用消息队列（如 RabbitMQ、Kafka）或者通过网络套接字进行通信，将对象从一个 JVM 发送到另一个。
  - 使用远程方法调用（RPC）
    - 可以使用远程方法调用框架，如 gRPC，来实现对象在不同 JVM 之间的传输。
  - 使用共享数据库或缓存
    - 将对象存储在共享数据库（如 MySQL、PostgreSQL）或共享缓存（如 Redis）中，让不同的 JVM 可以访问这些共享数据。
- 漏洞
  - 无法跨语言
    - Java 序列化目前只适用基于 Java 语言实现的框架，其它语言大部分都没有使用 Java 的序列化框架，也没有实现 Java 序列化这套协议。
  - 容易被攻击
    - Java 序列化是不安全的，我们知道对象是通过在 ObjectInputStream 上调用 readObject() 方法进行反序列化的，这个方法其实是一个神奇的构造器，它可以将类路径上几乎所有实现了 Serializable 接口的对象都实例化。这也就意味着，在反序列化字节流的过程中，该方法可以执行任意类型的代码，这是非常危险的。
  - 序列化后的流大大
    - 序列化后的二进制流大小能体现序列化的性能。序列化后的二进制数组越大，占用的存储空间就越多，存储硬件的成本就越高。如果我们是进行网络传输，则占用的带宽就更多，这时就会影响到系统的吞吐量。

② 同步/异步/阻塞/非阻塞

- 同步
  - 任务按顺序执行，必须等待当前操作完成后再执行下一步。
- 异步
  - 任务可以并行执行，无需等待操作完成即可继续执行其他任务。
- 阻塞
  - 线程被挂起，等待 I/O 操作完成才能继续执行。
- 非阻塞
  - 线程不会被挂起，立即返回结果（成功、失败或未完成）。

② Java 1.8

- Stream API
  - 提供了一种高效且易于使用的数据处理方式，特别适合集合对象的操作，如过滤、映射、排序等。
  - 案例
    - 过滤并收集满足条件的元素
      - List<String> filteredList = originalList.stream() .filter(s -> s.length() > 3) .collect(Collectors.toList());
    - 计算列表中所有数字的总和
      - int sum = numbers.stream().mapToInt(Integer::intValue).sum();
  - 并行API
    - ParallelStream
      - 将源数据分为多个子流对象进行多线程操作，然后将处理的结果再汇总为一个流对象
      - 底层是使用通用的 fork/join 池来实现，即将一个任务拆分成多个"小任务"并行计算，再把多个"小任务"的结果合并成总的计算结果
      - 对CPU密集型的任务来说，并行流使用Fork/JoinPool线程池，为每个CPU分配一个任务，这是非常有效率的
      - 如果任务是I/O密集的，并且任务数相对线程数比较大，那么直接用ParallelStream并不是很好的选择
- CompletableFuture
  - 由Java 8引入的，在Java8之前我们一般通过Future实现异步
  - 回调地狱
    - Future用于表示异步计算的结果，只能通过阻塞或者轮询的方式获取结果，而且不支持设置回调方法
    - 代码难以阅读
    - 难以维护
    - 错误处理复杂
    - 调试困难
  - CompletableFuture对Future进行了扩展，可以通过设置回调的方式处理计算结果，同时也支持组合操作，支持进一步的编排，同时一定程度解决了回调地狱的问题。

② 解释和编译

- 区别
  - 编译型语言
    - 在程序执行之前，整个源代码会被编译成机器码或者字节码，生成可执行文件。执行时直接运行编译后的代码，速度快，但跨平台性较差。
  - 解释型语言
    - 在程序执行时，逐行解释执行源代码，不生成独立的可执行文件。通常由解释器动态解释并执行代码，跨平台性好，但执行速度相对较慢。
- Java
  - 编译性
    - Java源代码首先被编译成字节码，JIT 会把编译过的机器码保存起来,以备下次使用。
  - 解释性
    - JVM中一个方法调用计数器，当累计计数大于一定值的时候，就使用JIT进行编译生成机器码文件。否则就是用解释器进行解释执行，然后字节码也是经过解释器进行解释运行的。

② 异常

- 类型
  - Error（错误）
    - 表示运行时环境的错误。错误是程序无法处理的严重问题，如系统崩溃、虚拟机错误、动态链接失败等。通常，程序不应该尝试捕获这类错误。
  - Exception（异常）
    - 非运行时异常
      - 这类异常在编译时期就必须被捕获或者声明抛出。它们通常是外部错误，如文件不存在、类未找到等。
    - 运行时异常
      - 这类异常包括运行时异常和错误。运行时异常由程序错误导致，如空指针访问、数组越界等。运行时异常是不需要在编译时强制捕获或声明的。
- 异常处理方法
  - try-catch语句块
    - 用于捕获并处理可能抛出的异常。try块中包含可能抛出异常的代码，catch块用于捕获并处理特定类型的异常。
  - throw语句
    - 用于手动抛出异常。可以根据需要在代码中使用throw语句主动抛出特定类型的异常。
  - throws关键字
    - 用于在方法声明中声明可能抛出的异常类型。如果一个方法可能抛出异常，但不想在方法内部进行处理，可以使用throws关键字将异常传递给调用者来处理。
  - finally块
    - 用于定义无论是否发生异常都会执行的代码块。通常用于释放资源，确保资源的正确关闭。

③ 设计模式

- 代理模式
  - 目的
    - 控制对对象的访问
  - 实现方式
    - 代理对象和被代理对象实现相同的接口
  - 使用场景
    - 远程代理、虚拟代理、保护代理、缓存代理
  - 关系
    - 代理对象和被代理对象是同一类对象
  - 关注点
    - 控制访问、增强功能
- 适配器模式
  - 目的
    - 使不兼容的接口能够协作
  - 实现方式
    - 适配器将被适配者的接口转换为目标接口
  - 使用场景
    - 集成第三方库、重用旧代码、统一接口
  - 关系
    - 适配器和被适配者是不同类对象
  - 关注点
    - 接口转换、兼容性

③ Native

- native关键字是Java语言中的一种声明，用于标记一个方法的实现将在外部定义。
- 允许Java代码调用外部的本地代码，即用C、C++或其他语言编写的代码。
  - 生成JNI头文件
    - 用 javah 工具生成对应的 C/C++ 头文件，头文件包含了 native 方法的 C/C++ 接口。
  - 编写本地代码
    - 使用C/C++编写本地方法的实现，并确保方法签名与生成的头文件中的原型匹配。
  - 编译本地代码
    - 将C/C++代码编译成动态链接库(DLL，在Windows上)，共享库(SO，在Linux上)。
  - 加载本地库
    - 在 Java 中通过 System.loadLibrary() 方法加载动态库，确保 JVM 可以正确解析和调用 native 方法。

③ 注解

- 本质是一个继承了Annotation的特殊接口，其具体实现类是Java运行时生成的动态代理类。通过反射获取注解时，返回的是Java运行时生成的动态代理对象。
- 作用域
  - 类级别作用域
    - 用于描述类的注解，通常放置在类定义的上面，可以用来指定类的一些属性，如类的访问级别、继承关系、注释等。
  - 方法级别作用域
    - 用于描述方法的注解，通常放置在方法定义的上面，可以用来指定方法的一些属性，如方法的访问级别、返回值类型、异常类型、注释等。
  - 字段级别作用域
    - 用于描述字段的注解，通常放置在字段定义的上面，可以用来指定字段的一些属性，如字段的访问级别、默认值、注释等。

③ I/O

- BIO(同步阻塞式 IO)
  - 每个客户端请求都由独立线程处理，I/O 操作（如 read()、write()）会导致线程阻塞，直到数据完成传输。
  - 适合对实时性要求不高的场景。
- NIO(同步非阻塞IO)
  - 使用 Selector 进行多路复用，一个线程可以管理多个 Channel，I/O 操作不会阻塞线程，提升了资源利用率。
  - 适合长连接、频繁 I/O 场景，避免为每个连接创建线程。
  - Channel
    - 数据通道，用于与底层 I/O 设备进行数据交互（类似于流 Stream，但支持双向操作）。
  - Buffer
    - 缓冲区，用于数据的临时存储，所有数据的读写都通过 Buffer 完成。数据总是从通道读取到缓冲区中，或者从缓冲区写入到通道中。
  - Selector
    - 多路复用器，监听多个 Channel 的 I/O 事件，基于操作系统底层的 select/poll/epoll 实现高效管理。
- AIO(异步非阻塞IO)
  - 使用操作系统的异步 I/O 能力，I/O 操作由 OS 负责完成，完成后通过回调 CompletionHandler 通知应用程序。
  - 适用于长时间等待和低延迟需求的应用。
- Netty(基于 NIO)
  - 基于 Reactor 模式，通过 多路复用、事件驱动和 异步非阻塞 机制，提升 I/O 处理效率，适用于高性能服务和分布式系统。

Presented with xmind